

Project Report: Assignment 3

March 25, 2026

Damaris Stephanie Ndjebayi, McGill University, School of Computer Science

Noé Vartanian, McGill University, School of Computer Science

Tarek Kawar, McGill University, School of Computer Science

Abstract:

The goal of this project is to identify the image that differs from the others within a group of five greyscale images. This Odd-One-Out visual reasoning task had as its main constraints the challenge of learning the meaningful relational features between images while respecting the imposed model size (≤ 25000 parameters). We approached this problem using a CNN where we embed each image into a compact feature space. For each image, the model compares its embedding with the remaining four images using multiple relational statistics, including mean differences, element-wise interactions, and distance-based features. Additional improvements, such as L2-normalized embeddings, label smoothing, and test-time augmentation (TTA), were incorporated to improve robustness and generalization. The final model contains 24181 trainable parameters and achieved a maximum public leaderboard score of 75.6%.

1. Introduction

The Odd-One-Out problem here is a visual reasoning task that requires identifying one element that does not belong within a group of four images. Each sample consists of five grayscale images, where four share similar structural characteristics, and one differs. The objective is to correctly identify the index of the outlier image.

This task is challenging for several reasons. Firstly, the task requires comparing multiple images within each group to determine which one differs from the others. This makes the problem challenging because the model must evaluate relationships between images rather than analyzing each image independently. Second, the dataset size is relatively small (3,000 training samples), making the problem prone to overfitting. Third, a constraint limits the number of trainable parameters to fewer than 25000, which prevents the use of larger & deeper convolutional architectures. Moreover, the datasets contain a small number of mislabeled samples, which introduces noise that can negatively affect the training.

To address these challenges, we created a 3-layer deep CNN that focuses on learning relationships between images rather than processing each image independently. A shared convolutional encoder is applied to all five images, ensuring parameter efficiency and consistent feature extraction. For each image candidate, the model computes a comparison between the candidate embedding and the average embedding of the remaining images. Additional statistical features, such as distance distributions and group spread metrics, help capture differences in variability among images.

Training was performed using the AdamW optimizer with cosine learning rate scheduling and label smoothing to improve generalization. Finally, test-time augmentation (TTA) was used during inference to stabilize predictions and improve performance without increasing model complexity.

2. Data Processing

The dataset consists of grayscale image groups stored in NumPy arrays with training data having shape (3000, 5, 32, 32). Each sample contains 5 grayscale images, and the labels indicate the index of the odd image. For the test data, we have the same structure without the labels. Each image has a resolution of 32x32 pixels, and all images are processed as single-channel inputs.

To evaluate model performance during development, the training data was split into training and validation sets using stratified sampling (stratified by labels), where the training set had 80% of the data, and the validation set had 20%. We fixed random seed 42 for reproducibility. The stratification ensured that all classes were proportionally represented in both splits.

To improve generalization and reduce overfitting, spatial data augmentation was applied during training. Importantly, augmentations were applied consistently across all five images in a group to preserve relative relationships. The augmentations included: horizontal and vertical flipping and random rotations (0°, 90°, 180°, or 270°). This design choice

ensured that the identity of the odd image remained valid after transformation, preventing artificial distortions in relational structure. An additional augmentation technique involved randomly permuting the order of the five images within each group during training. The corresponding labels were updated accordingly. This technique helped the model learn permutation-invariant reasoning and prevented reliance on positional bias.

Model selection was based on validation accuracy, with early stopping used to prevent overfitting. The best-performing model checkpoint on the validation set was retained for final training.

After determining optimal hyperparameters and training duration, a final model was retrained on the full dataset to maximize the use of available data before generating test predictions.

During inference, predictions were averaged across multiple augmented versions of each sample using test-time augmentation (TTA). The following augmentations were applied: random permutations of image order, random flips, random rotations, and number of TTA runs: 30. This approach significantly stabilized predictions and improved overall accuracy without increasing model size.

3. Method

a. Model architecture

We use a single end-to-end model that assigns a score to each of the 5 images in a set, representing how likely that image is to be the odd-one-out. Given one sample $X = \{x_0, \dots, x_4\}$, where each image $x_i \in \mathbb{R}^{1 \times 32 \times 32}$, the model first encodes each image independently using a shared lightweight CNN. This produces an embedding $z_i = f_\theta(x_i) \in \mathbb{R}^D$, with embedding dimension $D = 24$. To stabilize relational comparisons, each embedding is then L2-normalized: $\tilde{z}_i = \frac{z_i}{\|z_i\|_2}$.

This normalization reduces sensitivity to embedding magnitude and encourages the model to focus on relative geometric relationships between images rather than scale. For each candidate index i , we compare the normalized embedding $\tilde{z}_i$ to the remaining four embeddings and compute a scalar score s_i . The final prediction is the index with the largest score: $\hat{y} = \underset{i}{\operatorname{argmax}} (s_i)$.

Shared Encoder: Each of the five images is processed independently by the same encoder, which keeps the model permutation-agnostic and greatly reduces the number of trainable parameters. The encoder is composed of:

1. Conv2d $1 \rightarrow 12$, kernel size 4, padding 1	5. Conv2d $12 \rightarrow 24$, kernel size 4, padding 1	9. Conv2d $24 \rightarrow 32$, kernel size 4, padding 1
2. BatchNorm2d	6. BatchNorm2d	10. BatchNorm2d
3. GELU	7. GELU	11. GELU
4. MaxPool2d(2)	8. MaxPool2d(2)	12. AdaptiveAvgPool2d(1,1)
		13. Linear $32 \rightarrow 24$

This produces a 24-dimensional embedding for each image. The embedding is then L_2 -normalized, which stabilizes similarity and distance comparisons across images.

Candidate scoring: Let $z_i \in R^{24}$ denote the embedding of candidate image i , and let the remaining four embeddings be denoted as O_i . For each candidate, the model computes a relational feature vector based on the candidate and the other four images. Specifically, it includes:

- The candidate embedding z_i
- The mean of the other four embeddings (prototype)
- The standard deviation of the other four embeddings
- The absolute difference between the candidate and the prototype
- The elementwise product of the candidate and the prototype
- Four distance statistics between the candidate and the four others:
 - Mean distance, maximum distance, minimum distance, variance of distances
- The average pairwise distance among the other four images (group spread)

These features are concatenated into a vector of dimension: $5 \times 24 + 4 + 1 = 125$

And then this vector is passed through a lightweight MLP:

Linear 125 $\rightarrow$ 48	> GELU	> Dropout ($p = 0.15$)	> Linear 48 $\rightarrow$ 1
-----------------------------	--------	--------------------------	-----------------------------

This produces one scalar score for each candidate position. Repeating the procedure for all five images yields a final score vector of size 5.

b. Loss Function and Optimization

We train the model with cross-entropy loss over the 5 candidate scores. To reduce overconfidence and improve generalization, we use label smoothing with smoothing factor 0.05.

Optimization is performed with AdamW, using: learning rate: 5×10^{-3} and weight decay: 5×10^{-4} . We use a cosine annealing learning-rate schedule with Tmax equal to the total number of training epochs.

c. Hyperparameters and schedules

The Hyperparameters are described in the two following tables:

Hyperparameter	Value
Embedding dimension	24
Hidden dimension (scorer)	48
Dropout	0.15
Batch size	128
Optimizer	AdamW
Learning rate	5×10^{-3}

Hyperparameter	Value
Weight decay	5×10^{-4}
Loss	Cross-entropy
Label smoothing	0.05
Max epochs	75
Early stopping patience	15
Random seed	42

For model selection, the training set is split using a stratified 80/20 train-validation split. We train for at most 75 epochs and apply early stopping with patience 15, based on validation accuracy. The best checkpoint is selected according to validation performance.

After selecting the best epoch on the validation split, a final model is retrained on the full training set for exactly that number of epochs.

At inference time, we apply test-time augmentation (TTA). For each sample, we perform 30 stochastic passes, each with a random permutation and a shared spatial transformation. Scores are mapped back to the original image order before averaging. The final prediction is the class with the largest aggregated score.

d. Parameter limit (≤ 25000)

We ensured this in three ways:

- Shared encoder: the same CNN is reused for all five images, so parameters do not grow with the number of images.
- Small embedding size: the encoder outputs only 24 dimensions.
- Small scorer MLP: the candidate scoring network uses only one hidden layer of size 48.

The total number of trainable parameters is: 24181, which satisfies the required limit.

4. Results and Analysis

The learning curve below shows steady validation improvement plateauing as it reaches its peak performance at 62 epochs with a training validation gap of 0.0583, clearly illustrating the model’s stable convergence, indicating effective model learning without significant overfitting, further supported by early stopping measures. The final model achieved a public Kaggle score of 75.6% (standard: 75.0%, TTA (30): 76.2%), demonstrating strong generalization despite the parameter constraints. Table 2 represents an ablation study tracing clearly the modifications and how we reached the final tunings for the model. The largest single gain came from adding distance variance to distance stats, causing an increase in parameter usage to $\sim 24k$, increasing epoch(20-60) and patience(8-12), and adding TTA. This helped increase our score by $\sim 8\%$. After this big jump, a failed experiment applying mix-up augmentation and preimage normalization reduced our accuracy to 63%, as blending two groups destroyed the relational structure of the dataset. For our final model, we tuned the parameters, increasing the epoch to 75 and the patience to 15, finally adding L2-normalized embeddings.

The use of significant relational features and distance-based statistics significantly outperforms simpler models relying mainly on pairwise comparisons, emphasizing the importance of capturing central tendencies and variability. L2-normalized embeddings improved stability by ensuring comparisons focused on the relative structure instead of magnitude. Test-Time Augmentation with 30 stochastic passes reduced prediction variance across transformations, which further improved the robustness of the model. To improve generalization and prevent position bias, permutation-based augmentation was implemented during the training of the model. While label smoothing was introduced to reduce overconfidence and enhance performance for more general samples. The combination of relational feature engineering, augmentation strategies, and lightweight architecture produced strong results under tight constraints.

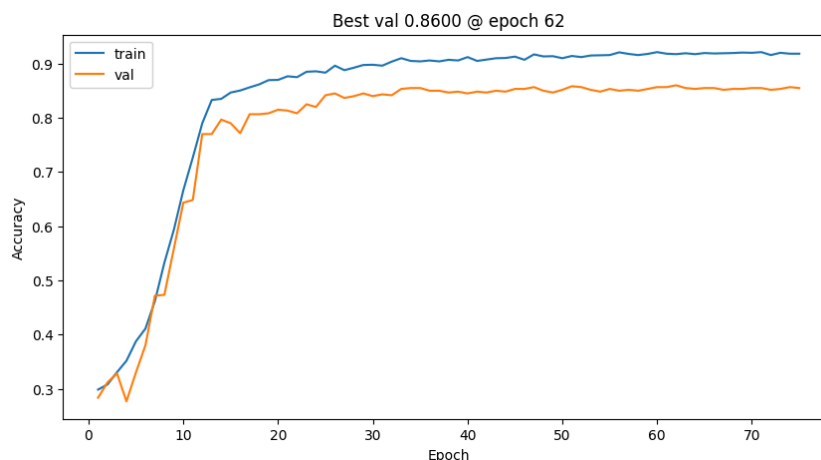


Table 2. Ablation study: effect of each modification on validation and public test accuracy.

Configuration	Val Acc	Public Acc
CNN encoder, 24-dim embeddings, Epoch (20), low parameter ~15k + computes embeddings, abs diff, product, distance stats: mean, max, min +MLP Scores each candidate + Augmentation: Horizontal flips +test different seeds (7, 42, 21) best = 42,	77.67%	61.6%
+ Augmentation: Vertical flip + rotation (0, 90, 180, 270 degrees) + Distance variance to distance stats + Higher epoch (60) + higher patience (12), seed set to 42 + TTA (set at 20) + parameters increased to ~24k	85%	69.9%
+ L2 normalization embedding + TTA set to 30 (Failed) +Per-image Norm + Mixup aug + increased dropout and decay	81%	~63%
+ Group spread features + Higher Epoch (75), Higher Patience (15) - Removed Mixup Augmentation - set dropout and weight decay to previous levels	86%	75.6%

5. Discussion and Conclusion

This project illustrates the possible effectiveness and strong performance of smaller models and lightweight architecture in relational visual learning tasks. The shared encoder, providing parameter efficiency and consistent feature extraction, along with the relational features, and TTA captured subtle differences while minimizing positional bias, helping the model identify the “odd-one-out”. With that said, we still faced many limitations, including a relatively small dataset restricting diversity and increasing the sensitivity and effects of mislabeled samples. The small parameter limit also forced us to simplify our model and forced us to rely on specific, customized relational features, hindering the model's ability to be repurposed to learn a more complex or abstract relation. To further this study, future work could explore residual encoder architectures to be applied to embeddings and k-fold cross validations for more reliable generalization estimates. Overall, this work demonstrates that a carefully engineered lightweight network can achieve strong performances on constrained tasks, evident by our model reaching a 75.6% public accuracy in a visual reasoning task setting constrained to a maximum of 25000 parameters.

Statement of Contributions:

All members worked on brainstorming, report writing, and coding.

Statement on the use of LLMs and other resources:

Large Language Models (LLMs) were used as a writing and editing aid to improve clarity, structure, and language quality of the report. The LLM was also used to assist with code organization and debugging suggestions. All modeling decisions, data analysis, experimental design, implementation, and interpretation of results were conducted by the authors. No external solutions or answers were copied directly, and all conclusions reflect the authors’ own understanding and work.